

BCS703 – CRYPTOGRAPHY AND NETWORK SECURITY

Module 2 and Module 3

Question Bank – Model Answers (10 / 15 Marks)

Department of CSE (AI & ML)

RNS Institute of Technology

Q. No.	Topic
1	Diffie–Hellman key exchange algorithm
2	Man-in-the-Middle attack on Diffie–Hellman
3	Diffie–Hellman numerical ($p = 23$, $g = 5$)
4	ECDH numerical ($n_A = 121$, $n_B = 203$)
5	Asymmetric encryption for authentication in symmetric key distribution; MITM
6	Cryptographic hash functions in message authentication
7	Key Distribution Centre (KDC)
8	Control vector encryption and decryption
9	X.509 certificate and PKI architecture
10	Schemes of public key distribution
11	Diffie–Hellman numerical ($p = 29$, $g = 2$)
12	Diffie–Hellman numerical ($p = 31$, $g = 3$)
13	Diffie–Hellman numerical ($p = 17$, $g = 3$)
14	ECDH numerical ($n_A = 5$, $n_B = 7$)
15	ECDH numerical ($n_A = 3$, $n_B = 9$)

Q.1. Explain the Diffie–Hellman key exchange algorithm in detail. Describe the key exchange process with a suitable example and neat diagram.

Answer:

Introduction

Diffie–Hellman (D-H) was the first public-key type scheme, proposed by Diffie and Hellman in 1976, for **key distribution only**. The purpose of the algorithm is to enable two users to exchange a secret key securely, which can then be used for subsequent encryption of messages. The algorithm itself is limited to the exchange of the keys.

Global (publicly known) elements

- A prime number q .
- An integer α that is a **primitive root of q** ($\alpha < q$).

Algorithm (users A and B wish to create a shared key)

1. Alice and Bob share the prime number q and the integer α , such that $\alpha < q$ and α is a primitive root of q .
2. Alice generates a private key X_A such that $X_A < q$. Bob generates a private key X_B such that $X_B < q$.
3. Alice calculates her public key $Y_A = \alpha^{X_A} \bmod q$. Bob calculates his public key $Y_B = \alpha^{X_B} \bmod q$.
4. Alice receives Bob's public key Y_B in plaintext, and Bob receives Alice's public key Y_A in plaintext.
5. Alice computes the secret key $K = (Y_B)^{X_A} \bmod q$. Bob computes the secret key $K = (Y_A)^{X_B} \bmod q$.

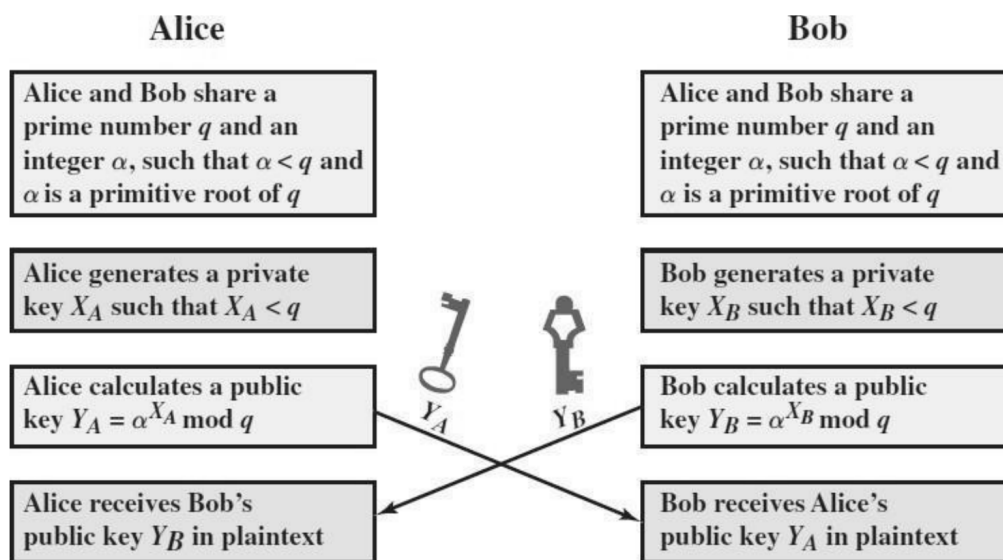


Figure: The Diffie–Hellman key exchange algorithm (from Module 2 notes). Last step: Alice computes $K = (Y_B)^{X_A} \bmod q$ and Bob computes $K = (Y_A)^{X_B} \bmod q$.

Why both sides get the same key

$K = (Y_B)^{X_A} \bmod q = (\alpha^{X_B})^{X_A} \bmod q = \alpha^{X_B X_A} \bmod q = (\alpha^{X_A})^{X_B} \bmod q = (Y_A)^{X_B} \bmod q$. Hence A and B arrive at the same secret key K without ever transmitting it. An attacker knows only q , α , Y_A and Y_B but not the private keys X_A and X_B .

Example (from the notes)

Example:

- Let's take $q = 353$ and $\alpha = 3$ (a primitive root of 353).
- A and B select private keys $X_A = 97$ and $X_B = 233$, respectively.
- Each computes its public key:

A computes $Y_A = 3^{97} \bmod 353 = 40$.

B computes $Y_B = 3^{233} \bmod 353 = 248$.
- After they exchange public keys, each can compute the common secret key:

A computes $K = (Y_B)^{X_A} \bmod 353 = 248^{97} \bmod 353 = 160$.

B computes $K = (Y_A)^{X_B} \bmod 353 = 40^{233} \bmod 353 = 160$.
- An attacker would have available the following information but not Private Keys:

$$q = 353, \alpha = 3, Y_A = 40, Y_B = 248$$

Figure: Diffie-Hellman example with $q = 353$ and $\alpha = 3$ (Module 2 notes).

Verification of the example: $q = 353$, $\alpha = 3$, $X_A = 97$, $X_B = 233$.

- A computes $Y_A = 3^{97} \bmod 353 = \mathbf{40}$; B computes $Y_B = 3^{233} \bmod 353 = \mathbf{248}$.
- A computes $K = (Y_B)^{X_A} \bmod 353 = 248^{97} \bmod 353 = \mathbf{160}$.
- B computes $K = (Y_A)^{X_B} \bmod 353 = 40^{233} \bmod 353 = \mathbf{160}$.
- Shared secret key = 160. The attacker has only $q = 353$, $\alpha = 3$, $Y_A = 40$, $Y_B = 248$.

Conclusion

The security of the scheme rests on the difficulty of finding the private key from the public key, since only public values are exchanged. However, the public keys are exchanged in plaintext without authentication, which makes the scheme open to the Man-in-the-Middle attack (Q.2).

Q.2. With a neat diagram, explain the Man-in-the-Middle attack on the Diffie-Hellman key exchange and discuss how it can be prevented.

Answer:

Man-in-the-Middle attack

Suppose Alice and Bob wish to exchange keys and **Darth** is the adversary. The attack proceeds as follows:

1. Darth prepares for the attack by generating two random private keys X_{D1} and X_{D2} and computing the corresponding public keys Y_{D1} and Y_{D2} .
2. Alice transmits Y_A to Bob.
3. Darth intercepts Y_A and transmits Y_{D1} to Bob. Darth also calculates $K2 = (Y_A)^{X_{D2}} \bmod q$.
4. Bob receives Y_{D1} and calculates $K1 = (Y_{D1})^{X_B} \bmod q$.
5. Bob transmits Y_B to Alice.
6. Darth intercepts Y_B and transmits Y_{D2} to Alice. Darth calculates $K1 = (Y_B)^{X_{D1}} \bmod q$.
7. Alice receives Y_{D2} and calculates $K2 = (Y_{D2})^{X_A} \bmod q$.

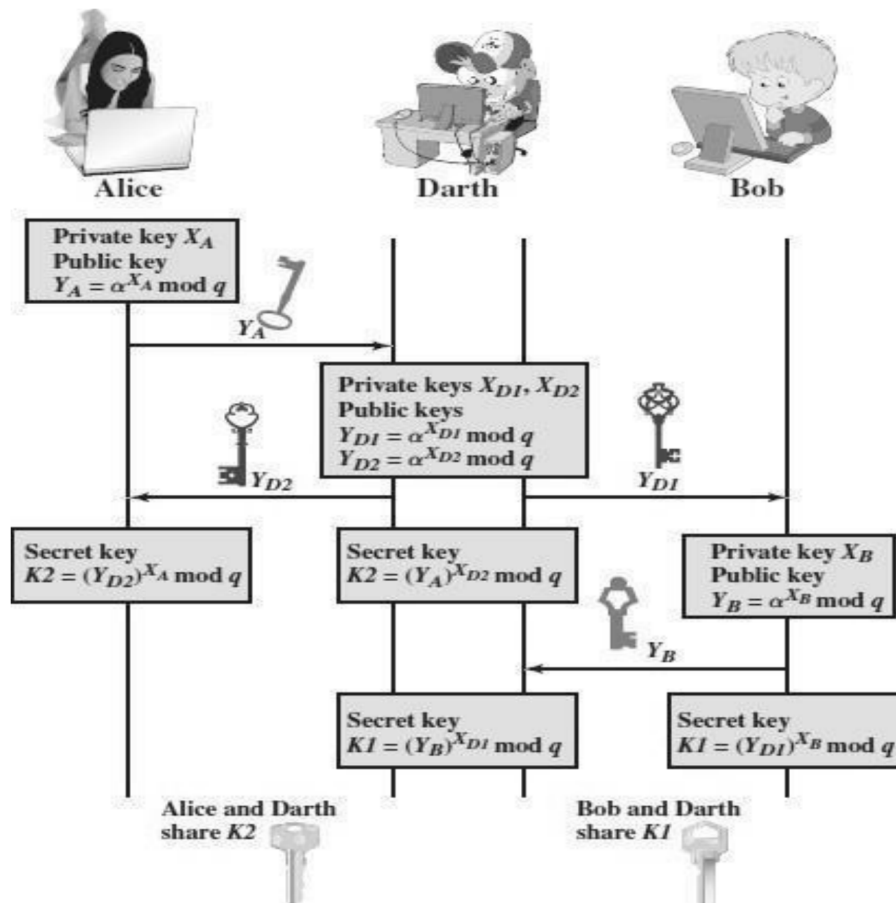


Figure: Man-in-the-Middle attack on Diffie-Hellman key exchange (Module 2 notes)

At this point Bob and Alice think that they share a secret key, but instead **Bob and Darth share the secret key K1** and **Alice and Darth share the secret key K2**.

Effect on later communication

1. Alice sends an encrypted message M : $E(K2, M)$.
2. Darth intercepts the encrypted message and decrypts it to recover M .

3. Darth sends Bob $E(K1, M)$ or $E(K1, M')$, where M' is any message.
- In the first case, Darth simply wants to **eavesdrop** on the communication without altering it.
 - In the second case, Darth wants to **modify** the message going to Bob.

Why the attack succeeds

The D-H protocol as described exchanges the public values Y_A and Y_B in plaintext and gives no way for a receiver to confirm *who* a public value came from. Darth can therefore substitute his own values, and neither party can detect it.

Prevention (using the authentication techniques of Module 3)

The attack is defeated by **authenticating the public values** that are exchanged, so that the value received is known to belong to the claimed sender:

- **Public-key certificates (X.509):** a certificate binds an identity to a public key and is signed by a trusted Certification Authority (CA). Anyone who knows the CA's public key can verify that the certificate is genuine and not counterfeit, so Darth cannot pass off his own value as Alice's or Bob's.
- **Digital signatures:** the sender signs the hash of the message (here, the public value) with its private key; the receiver verifies it with the sender's public key, which gives authentication and integrity (Figure 11.4a of Module 3).
- **Authenticated exchange with nonces:** using the scheme of confidentiality and authentication (Q.5), the parties encrypt with each other's public keys and exchange nonces N_1, N_2 , so each side is assured that the correspondent is the genuine party.

With authenticated public values, Darth's substituted values Y_{D1} and Y_{D2} are rejected and the two parties obtain a key shared only between themselves.

Q.3. Let $p = 23$ and $g = 5$. If the private keys of users A and B are 6 and 15, respectively, use the Diffie–Hellman key exchange algorithm to calculate the public keys P_A , P_B , and the common shared secret key, showing the necessary steps.

Answer:

Formulae: public key = $g^{\text{private}} \bmod p$; shared key $K = (\text{other user's public key})^{\text{own private key}} \bmod p$.

Given: prime $q (p) = 23$, primitive root $\alpha (g) = 5$, private key of A (X_A) = 6, private key of B (X_B) = 15.

Step 1: Public key of A ($Y_A = \alpha^{X_A} \bmod q$)

$$Y_A = 5^6 \bmod 23$$

$$5^1 \bmod 23 = 5$$

$$5^2 = (5)^2 \bmod 23 = 2$$

$$5^4 = (2)^2 \bmod 23 = 4$$

Since $6 = 4 + 2$:

$$5^6 = 4 \times 2 \pmod{23}$$

$$4 \times 2 = 8 \bmod 23 = 8$$

$$Y_A = 8$$

Step 2: Public key of B ($Y_B = \alpha^{X_B} \bmod q$)

$$Y_B = 5^{15} \bmod 23$$

$$5^1 \bmod 23 = 5$$

$$5^2 = (5)^2 \bmod 23 = 2$$

$$5^4 = (2)^2 \bmod 23 = 4$$

$$5^8 = (4)^2 \bmod 23 = 16$$

Since $15 = 8 + 4 + 2 + 1$:

$$5^{15} = 16 \times 4 \times 2 \times 5 \pmod{23}$$

$$16 \times 4 = 64 \bmod 23 = 18$$

$$18 \times 2 = 36 \bmod 23 = 13$$

$$13 \times 5 = 65 \bmod 23 = 19$$

$$Y_B = 19$$

Step 3: Exchange. A sends $Y_A = 8$ to B and B sends $Y_B = 19$ to A (in plaintext).

Step 4: Shared secret key computed by A ($K = (Y_B)^{X_A} \bmod q$)

$$K = 19^6 \bmod 23$$

$$19^1 \bmod 23 = 19$$

$$19^2 = (19)^2 \bmod 23 = 16$$

$$19^4 = (16)^2 \bmod 23 = 3$$

Since $6 = 4 + 2$:

$$19^6 = 3 \times 16 \pmod{23}$$

$$3 \times 16 = 48 \bmod 23 = 2$$

$$K = 2$$

Step 5: Shared secret key computed by B ($K = (Y_A)^{X_B} \bmod q$)

$$K = 8^{15} \bmod 23$$

$$8^1 \bmod 23 = 8$$

$$8^2 = (8)^2 \bmod 23 = 18$$

$$8^4 = (18)^2 \bmod 23 = 2$$

$$8^8 = (2)^2 \bmod 23 = 4$$

Since $15 = 8 + 4 + 2 + 1$:

$$8^{15} = 4 \times 2 \times 18 \times 8 \pmod{23}$$

$$4 \times 2 = 8 \bmod 23 = 8$$

$$8 \times 18 = 144 \bmod 23 = 6$$

$$6 \times 8 = 48 \bmod 23 = 2$$

K = 2

Quantity	Value
Public key of A, Y_A	8
Public key of B, Y_B	19
Shared secret key K (computed by A)	2
Shared secret key K (computed by B)	2

Both users obtain the same secret key **K = 2**. An attacker sees only $q = 23$, $\alpha = 5$, $Y_A = 8$, $Y_B = 19$ but not the private keys.

Q.4. Apply the Elliptic Curve Diffie–Hellman (ECDH) key exchange procedure for $p = 211$, $E_p(0, -4)$, $G = (2, 2)$, $n_A = 121$, and $n_B = 203$. Find P_A , P_B , and the common shared secret key, showing the necessary steps.

Answer:

ECDH procedure (from the notes)

1. Pick a large integer q (prime or of the form 2^m) and curve parameters a, b , defining the elliptic group $E_q(a, b)$. Pick a base point $G = (x_1, y_1)$ of large order n . These are known to all participants.
 2. A selects an integer $n_A < n$ (private key) and computes the public key $P_A = n_A \times G$, a point in $E_q(a, b)$.
 3. B similarly selects a private key n_B and computes the public key $P_B = n_B \times G$.
 4. A generates the secret key $k = n_A \times P_B$; B generates the secret key $k = n_B \times P_A$.
- Both are equal because $n_A \times P_B = n_A \times (n_B \times G) = n_B \times (n_A \times G) = n_B \times P_A$.

Solution

Given: elliptic curve $E_{211}(0, -4)$, i.e. $y^2 \equiv x^3 - 4 \pmod{211}$ with $a = 0$, $b = -4$, $q = p = 211$, base point $G = (2, 2)$. Private keys: $n_A = 121$, $n_B = 203$.

Check that G lies on the curve: $y^2 = 2^2 = 4$ and $x^3 - 4 = 8 - 4 = 4$. Since both sides are equal, $G = (2, 2)$ is a valid point.

Point addition rules used (mod 211, $a = 0$):

- For $P = (x_1, y_1)$, $Q = (x_2, y_2)$, $P \neq Q$: $\lambda = (y_2 - y_1)/(x_2 - x_1) \pmod{211}$.
- For doubling ($P = Q$): $\lambda = (3x_1^2 + a)/(2y_1) \pmod{211}$, with $a = 0$.
- Then $x_3 = \lambda^2 - x_1 - x_2 \pmod{211}$ and $y_3 = \lambda(x_1 - x_3) - y_1 \pmod{211}$.
- Scalar multiplication kG is done by the double-and-add method: write k in binary, start from G (leading bit); for every next bit, double the current point and, if the bit is 1, add G .

Step 1: Public key of A, $P_A = n_A \times G = 121G$. Binary of 121 = 1111001.

Step	Operation	$\lambda \pmod{211}$	$x_3 = \lambda^2 - x_1 - x_2$	$y_3 = \lambda(x_1 - x_3) - y_1$	Result
1	$2G = 2 \times 1G$ (2, 2) doubled	$(3 \cdot 2^2)/(2 \cdot 2) = 12/4 \equiv 12/4 \equiv 3$	$3^2 - 2 - 2 = 5 \equiv 5$	$3(2 - 5) - 2 = -11 \equiv 200$	$2G = (5, 200)$
2	$3G = 2G + G$ (5, 200) + (2, 2)	$(2 - 200)/(2 - 5) = -198/-3 \equiv 13/208 \equiv 66$	$66^2 - 5 - 2 = 4349 \equiv 129$	$66(5 - 129) - 200 = -8384 \equiv 56$	$3G = (129, 56)$
3	$6G = 2 \times 3G$ (129, 56) doubled	$(3 \cdot 129^2)/(2 \cdot 56) = 49923/112 \equiv 127/112 \equiv 52$	$52^2 - 129 - 129 = 2446 \equiv 125$	$52(129 - 125) - 56 = 152 \equiv 152$	$6G = (125, 152)$
4	$7G = 6G + G$ (125, 152) + (2, 2)	$(2 - 152)/(2 - 125) = -150/-123 \equiv 61/88 \equiv 99$	$99^2 - 125 - 2 = 9674 \equiv 179$	$99(125 - 179) - 152 = -5498 \equiv 199$	$7G = (179, 199)$
5	$14G = 2 \times 7G$ (179, 199) doubled	$(3 \cdot 179^2)/(2 \cdot 199) = 96123/398 \equiv 118/187 \equiv 83$	$83^2 - 179 - 179 = 6531 \equiv 201$	$83(179 - 201) - 199 = -2025 \equiv 85$	$14G = (201, 85)$
6	$15G = 14G + G$ (201, 85) + (2, 2)	$(2 - 85)/(2 - 201) = -83/-199 \equiv 128/12 \equiv 81$	$81^2 - 201 - 2 = 6358 \equiv 28$	$81(201 - 28) - 85 = 13928 \equiv 2$	$15G = (28, 2)$

Step	Operation	$\lambda \pmod{211}$	$x_3 = \lambda^2 - x_1 - x_2$	$y_3 = \lambda(x_1 - x_2) - y_1$	Result
7	$30G = 2 \times 15G$ (28, 2) doubled	$(3 \cdot 28^2)/(2 \cdot 2) = 2352/4 \equiv 31/4 \equiv \mathbf{166}$	$166^2 - 28 - 28 = 27500 \equiv 70$	$166(28 - 70) - 2 = -6974 \equiv 200$	30G = (70, 200)
8	$60G = 2 \times 30G$ (70, 200) doubled	$(3 \cdot 70^2)/(2 \cdot 200) = 14700/400 \equiv 141/189 \equiv \mathbf{195}$	$195^2 - 70 - 70 = 37885 \equiv 116$	$195(70 - 116) - 200 = -9170 \equiv 114$	60G = (116, 114)
9	$120G = 2 \times 60G$ (116, 114) doubled	$(3 \cdot 116^2)/(2 \cdot 114) = 40368/228 \equiv 67/17 \equiv \mathbf{66}$	$66^2 - 116 - 116 = 4124 \equiv 115$	$66(116 - 115) - 114 = -48 \equiv 163$	120G = (115, 163)
10	$121G = 120G + G$ (115, 163) + (2, 2)	$(2 - 163)/(2 - 115) = -161/-113 \equiv 50/98 \equiv \mathbf{134}$	$134^2 - 115 - 2 = 17839 \equiv 115$	$134(115 - 115) - 163 = -163 \equiv 48$	121G = (115, 48)

$P_A = 121G = (115, 48)$

Step 2: Public key of B, $P_B = n_B \times G = 203G$. Binary of 203 = 11001011.

Step	Operation	$\lambda \pmod{211}$	$x_3 = \lambda^2 - x_1 - x_2$	$y_3 = \lambda(x_1 - x_2) - y_1$	Result
1	$2G = 2 \times 1G$ (2, 2) doubled	$(3 \cdot 2^2)/(2 \cdot 2) = 12/4 \equiv 12/4 \equiv \mathbf{3}$	$3^2 - 2 - 2 = 5 \equiv 5$	$3(2 - 5) - 2 = -11 \equiv 200$	2G = (5, 200)
2	$3G = 2G + G$ (5, 200) + (2, 2)	$(2 - 200)/(2 - 5) = -198/-3 \equiv 13/208 \equiv \mathbf{66}$	$66^2 - 5 - 2 = 4349 \equiv 129$	$66(5 - 129) - 200 = -8384 \equiv 56$	3G = (129, 56)
3	$6G = 2 \times 3G$ (129, 56) doubled	$(3 \cdot 129^2)/(2 \cdot 56) = 49923/112 \equiv 127/112 \equiv \mathbf{52}$	$52^2 - 129 - 129 = 2446 \equiv 125$	$52(129 - 125) - 56 = 152 \equiv 152$	6G = (125, 152)
4	$12G = 2 \times 6G$ (125, 152) doubled	$(3 \cdot 125^2)/(2 \cdot 152) = 46875/304 \equiv 33/93 \equiv \mathbf{48}$	$48^2 - 125 - 125 = 2054 \equiv 155$	$48(125 - 155) - 152 = -1592 \equiv 96$	12G = (155, 96)
5	$24G = 2 \times 12G$ (155, 96) doubled	$(3 \cdot 155^2)/(2 \cdot 96) = 72075/192 \equiv 124/192 \equiv \mathbf{49}$	$49^2 - 155 - 155 = 2091 \equiv 192$	$49(155 - 192) - 96 = -1909 \equiv 201$	24G = (192, 201)
6	$25G = 24G + G$ (192, 201) + (2, 2)	$(2 - 201)/(2 - 192) = -199/-190 \equiv 12/21 \equiv \mathbf{91}$	$91^2 - 192 - 2 = 8087 \equiv 69$	$91(192 - 69) - 201 = 10992 \equiv 20$	25G = (69, 20)
7	$50G = 2 \times 25G$ (69, 20) doubled	$(3 \cdot 69^2)/(2 \cdot 20) = 14283/40 \equiv 146/40 \equiv \mathbf{183}$	$183^2 - 69 - 69 = 33351 \equiv 13$	$183(69 - 13) - 20 = 10228 \equiv 100$	50G = (13, 100)
8	$100G = 2 \times 50G$ (13, 100) doubled	$(3 \cdot 13^2)/(2 \cdot 100) = 507/200 \equiv 85/200 \equiv \mathbf{69}$	$69^2 - 13 - 13 = 4735 \equiv 93$	$69(13 - 93) - 100 = -5620 \equiv 77$	100G = (93, 77)
9	$101G = 100G + G$ (93, 77) + (2, 2)	$(2 - 77)/(2 - 93) = -75/-91 \equiv 136/120 \equiv \mathbf{184}$	$184^2 - 93 - 2 = 33761 \equiv 1$	$184(93 - 1) - 77 = 16851 \equiv 182$	101G = (1, 182)
10	$202G = 2 \times 101G$ (1, 182) doubled	$(3 \cdot 1^2)/(2 \cdot 182) = 3/364 \equiv 3/153 \equiv \mathbf{120}$	$120^2 - 1 - 1 = 14398 \equiv 50$	$120(1 - 50) - 182 = -6062 \equiv 57$	202G = (50, 57)
11	$203G = 202G + G$ (50, 57) + (2, 2)	$(2 - 57)/(2 - 50) = -55/-48 \equiv 156/163 \equiv \mathbf{155}$	$155^2 - 50 - 2 = 23973 \equiv 130$	$155(50 - 130) - 57 = -12457 \equiv 203$	203G = (130, 203)

$P_B = 203G = (130, 203)$

Step 3: A and B exchange P_A and P_B .

Step 4: Secret key computed by A, $K = n_A \times P_B = 121 \times (130, 203)$. (Binary of 121 = 1111001; the base point for double-and-add is now P_B .)

Step	Operation	$\lambda \pmod{211}$	$x_3 = \lambda^2 - x_1 - x_2$	$y_3 = \lambda(x_1 - x_3) - y_1$	Result
1	$2P_B = 2 \times 1P_B$ (130, 203) doubled	$(3 \cdot 130^2)/(2 \cdot 203) = 50700/406 \equiv 60/195 \equiv 49$	$49^2 - 130 - 130 = 2141 \equiv 31$	$49(130 - 31) - 203 = 4648 \equiv 6$	$2P_B = (31, 6)$
2	$3P_B = 2P_B + P_B$ (31, 6) + (130, 203)	$(203-6)/(130-31) = 197/99 \equiv 197/99 \equiv 132$	$132^2 - 31 - 130 = 17263 \equiv 172$	$132(31 - 172) - 6 = -18618 \equiv 161$	$3P_B = (172, 161)$
3	$6P_B = 2 \times 3P_B$ (172, 161) doubled	$(3 \cdot 172^2)/(2 \cdot 161) = 88752/322 \equiv 132/111 \equiv 24$	$24^2 - 172 - 172 = 232 \equiv 21$	$24(172 - 21) - 161 = 3463 \equiv 87$	$6P_B = (21, 87)$
4	$7P_B = 6P_B + P_B$ (21, 87) + (130, 203)	$(203-87)/(130-21) = 116/109 \equiv 116/109 \equiv 3$	$3^2 - 21 - 130 = -142 \equiv 69$	$3(21 - 69) - 87 = -231 \equiv 191$	$7P_B = (69, 191)$
5	$14P_B = 2 \times 7P_B$ (69, 191) doubled	$(3 \cdot 69^2)/(2 \cdot 191) = 14283/382 \equiv 146/171 \equiv 28$	$28^2 - 69 - 69 = 646 \equiv 13$	$28(69 - 13) - 191 = 1377 \equiv 111$	$14P_B = (13, 111)$
6	$15P_B = 14P_B + P_B$ (13, 111) + (130, 203)	$(203-111)/(130-13) = 92/117 \equiv 92/117 \equiv 8$	$8^2 - 13 - 130 = -79 \equiv 132$	$8(13 - 132) - 111 = -1063 \equiv 203$	$15P_B = (132, 203)$
7	$30P_B = 2 \times 15P_B$ (132, 203) doubled	$(3 \cdot 132^2)/(2 \cdot 203) = 52272/406 \equiv 155/195 \equiv 109$	$109^2 - 132 - 132 = 11617 \equiv 12$	$109(132 - 12) - 203 = 12877 \equiv 6$	$30P_B = (12, 6)$
8	$60P_B = 2 \times 30P_B$ (12, 6) doubled	$(3 \cdot 12^2)/(2 \cdot 6) = 432/12 \equiv 10/12 \equiv 36$	$36^2 - 12 - 12 = 1272 \equiv 6$	$36(12 - 6) - 6 = 210 \equiv 210$	$60P_B = (6, 210)$
9	$120P_B = 2 \times 60P_B$ (6, 210) doubled	$(3 \cdot 6^2)/(2 \cdot 210) = 108/420 \equiv 108/209 \equiv 157$	$157^2 - 6 - 6 = 24637 \equiv 161$	$157(6 - 161) - 210 = -24545 \equiv 142$	$120P_B = (161, 142)$
10	$121P_B = 120P_B + P_B$ (161, 142) + (130, 203)	$(203-142)/(130-161) = 61/-31 \equiv 61/180 \equiv 175$	$175^2 - 161 - 130 = 30334 \equiv 161$	$175(161 - 161) - 142 = -142 \equiv 69$	$121P_B = (161, 69)$

K = (161, 69)

Step 5: Secret key computed by B, $K = n_B \times P_A = 203 \times (115, 48)$. The same double-and-add procedure (binary of 203 = 11001011) gives the successive points:

$2P_A = (2, 2)$; $3P_A = (156, 10)$; $6P_A = (129, 56)$; $12P_A = (125, 152)$; $24P_A = (155, 96)$; $25P_A = (145, 127)$; $50P_A = (69, 20)$; $100P_A = (13, 100)$; $101P_A = (209, 153)$; $202P_A = (1, 182)$; $203P_A = (161, 69)$

K = (161, 69)

This is the same point, because $n_A \times P_B = n_A \times (n_B \times G) = n_B \times (n_A \times G) = n_B \times P_A$.

Quantity	Value
$P_A = n_A G$	(115, 48)
$P_B = n_B G$	(130, 203)
$K = n_A \times P_B$	(161, 69)
$K = n_B \times P_A$	(161, 69)

Common shared secret key $K = (161, 69)$. (Usually the x-coordinate, 161, is used as the key material.)

Q.5. Explain how asymmetric encryption is used to achieve authentication during symmetric key distribution. Describe the Man-in-the-Middle (MITM) attack and its prevention with a suitable diagram.

Answer:

Introduction

Once public keys have been distributed or have become accessible, secure communication that thwarts eavesdropping, tampering, or both, is possible. Public-key encryption therefore provides for the distribution of secret keys to be used for conventional (symmetric) encryption.

(a) Simple secret key distribution

1. A generates a public/private key pair $\{PU_a, PR_a\}$ and transmits a message to B consisting of PU_a and an identifier of A, ID_A .
2. B generates a secret key K_s and transmits it to A, encrypted with A's public key.
3. A computes $D(PR_a, E(PU_a, K_s))$ to recover the secret key. Because only A can decrypt the message, only A and B will know the identity of K_s .
4. A discards PU_a and PR_a and B discards PU_a .

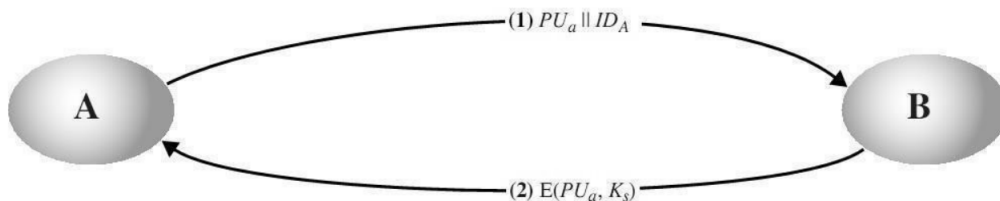


Figure: Simple secret key distribution using public-key encryption (Module 3 notes)

Man-in-the-Middle attack on the simple scheme

Here a third party can **intercept messages and then either relay the intercepted message or substitute another message**. Such an attack is known as a **man-in-the-middle attack**. For example, Darth intercepts A's message $PU_a || ID_A$ and sends his own public key in its place; B then encrypts K_s with Darth's key, so Darth can recover K_s and pass it on to A. A and B believe they share a secret key, but Darth knows it too. The scheme gives secrecy but no *authentication* of the parties, as in the Diffie–Hellman attack of Q.2.

(b) Secret key distribution with confidentiality and authentication (prevention)

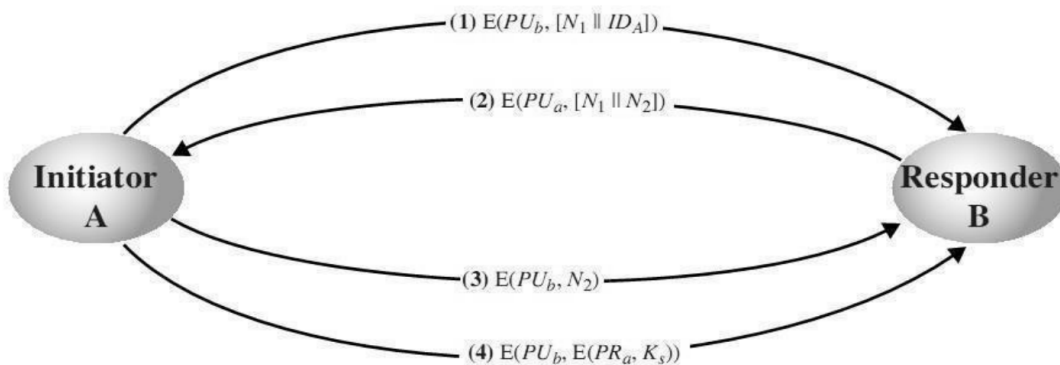


Figure: Secret key distribution with confidentiality and authentication (Module 3 notes)

It is assumed that A and B already know each other's public keys. The steps are:

1. A uses B's public key to encrypt a message to B containing an identifier of A (ID_A) and a nonce (N_1), which is used to identify this transaction uniquely: $E(PU_b, [N_1 \parallel ID_A])$.
2. B sends a message to A encrypted with PU_a containing A's nonce N_1 and a new nonce N_2 generated by B: $E(PU_a, [N_1 \parallel N_2])$. Because only B could have decrypted message (1), the presence of N_1 in message (2) assures A that the correspondent is B.
3. A returns N_2 encrypted using B's public key, $E(PU_b, N_2)$, to assure B that its correspondent is A.
4. A selects a secret key K_s and sends $M = E(PU_b, E(PR_a, K_s))$ to B. Encryption with B's public key ensures that only B can read it; encryption with A's private key ensures that only A could have sent it.
5. B computes $D(PU_a, D(PR_b, M))$ to recover the secret key K_s .

How authentication is achieved and MITM is prevented

- **Authentication by public-key decryption:** only B holds PR_b , so only B can decrypt message (1); only A holds PR_a , so only A can produce $E(PR_a, K_s)$.
- **Nonces** N_1 and N_2 tie each reply to the request that caused it, proving the correspondent is genuine and not a replay or an impostor.
- **Confidentiality:** K_s is sent inside an encryption with B's public key, so an interceptor cannot read it. An interceptor also cannot substitute a key of his own, because he cannot produce A's private-key encryption of it.

(c) Hybrid scheme

Another way is a hybrid approach that retains the use of a key distribution center (KDC) sharing a secret master key with each user and distributing session keys encrypted with the master key. A public-key scheme is used to distribute the master keys; this public-key layer provides a secure and efficient means of distributing master keys.

Note: the authenticity of the public keys themselves is ensured by the distribution methods of Q.10 (certificates, public-key authority).

Q.6. Explain the role of cryptographic hash functions in message authentication with a neat diagram. Describe how message integrity and authenticity are verified.

Answer:

Introduction

The cryptographic hash function is one of the most versatile algorithms, widely used in security applications and Internet protocols. It provides data integrity and authentication, and supports mechanisms such as digital signatures and password protection.

Message authentication

Message authentication verifies that a message has **not been modified, inserted, deleted, or replayed**, and also assures that the sender's identity is genuine. When a hash function is used for authentication, the resulting value is called a **message digest**.

- The **sender** computes a hash value from the message and sends both the message and its hash.
- The **receiver** recomputes the hash from the received message and compares it with the received hash.
- A **mismatch** indicates alteration of the message or of the hash (Figure 11.2a).

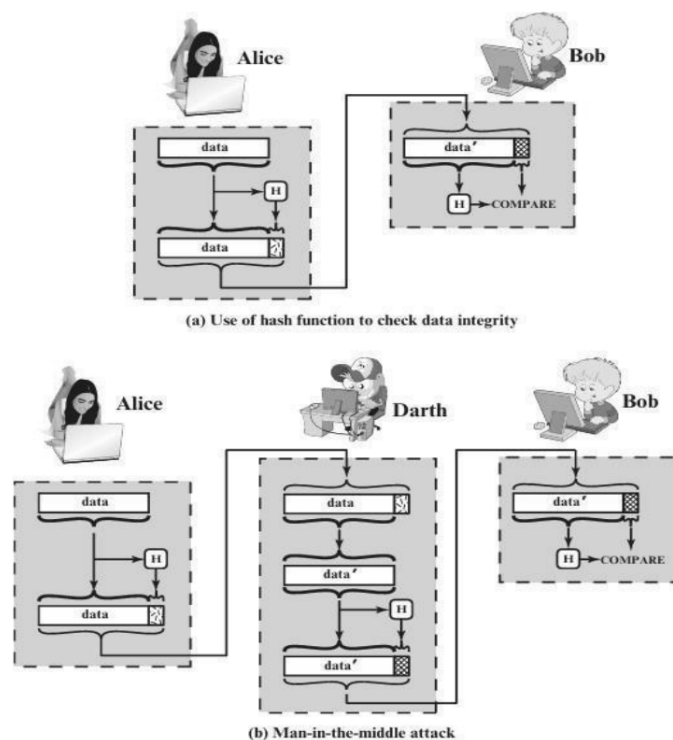


Figure 11.2 Attack Against Hash Function

Figure 11.2: (a) Use of a hash function to check data integrity; (b) Man-in-the-middle attack (Module 3 notes)

If an attacker (Darth) alters the message and recomputes a new hash, the receiver might be **fooled** (Figure 11.2b). Hence **the hash value must be protected**.

Methods of using hash codes for authentication (Figure 11.3)

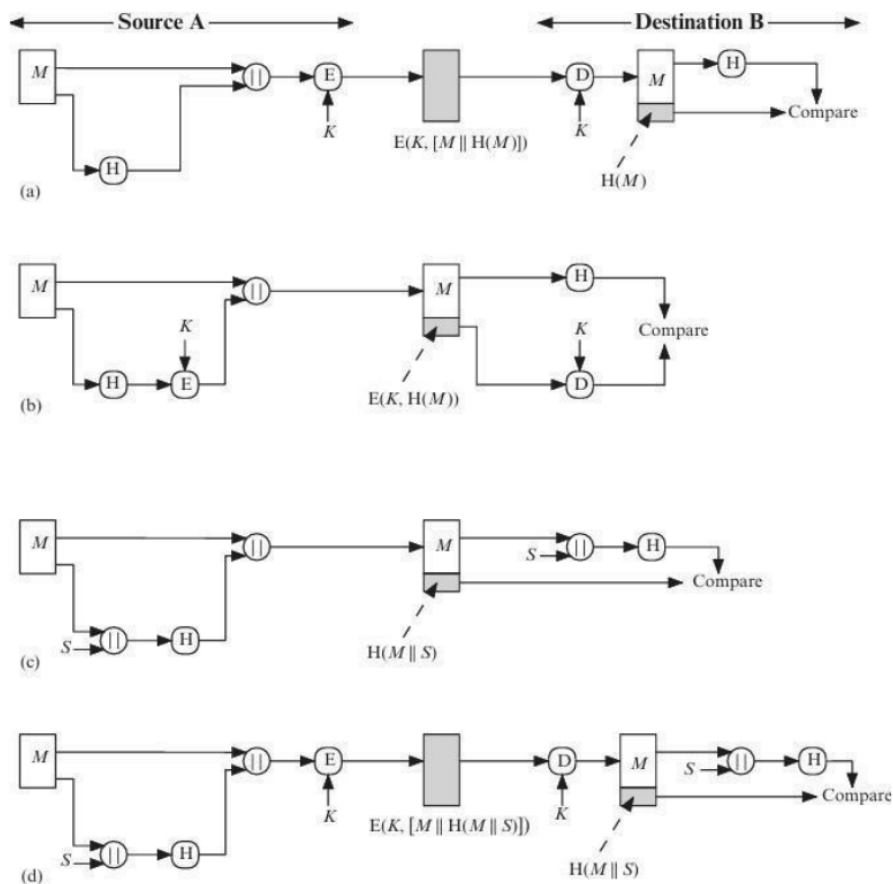


Figure 11.3 Simplified Examples of the Use of a Hash Function for Message Authentication

Figure 11.3: Simplified examples of the use of a hash function for message authentication (Module 3 notes)

Method	Working	Provides
(a)	Encrypt (message + hash), $E(K, [M \parallel H(M)])$, using symmetric encryption; the receiver decrypts, recomputes $H(M)$ and compares.	Authentication and confidentiality
(b)	Encrypt only the hash code, $E(K, H(M))$, and send it with M . The receiver recomputes $H(M)$, decrypts the received code and compares.	Authentication with less computation, when confidentiality is not required
(c)	Use a shared secret value S without encryption: the sender computes $H(M \parallel S)$ and sends it with M . The receiver recomputes $H(M \parallel S)$ with the same S and compares. S is never transmitted.	Authentication; tampering prevented as S is unknown to the attacker
(d)	Method (c) plus encryption of (message + hash): $E(K, [M \parallel H(M \parallel S)])$.	Added confidentiality

Method (b) is often preferred because encryption of the entire message is computationally expensive.

Reasons to avoid full encryption:

- Encryption software is slow for frequent small messages.
- Hardware cost adds up for all nodes.
- Hardware initialization overhead is high for small data.
- Encryption algorithms may be patent protected.

Message Authentication Code (MAC)

A MAC (keyed hash function) is used when two parties share a secret key K : $\text{MAC} = F(K, M)$. It verifies both integrity and authenticity, since an attacker cannot alter the message or the MAC without knowing K . Combining hashing and encryption, e.g. $E(K, H(M))$, acts as a MAC. MACs are more

efficient than encryption algorithms.

Digital signatures

A digital signature is similar to a MAC but uses public-key encryption. The hash value of the message is encrypted with the **sender's private key**; anyone with the sender's **public key** can verify it. It ensures integrity, authentication and non-repudiation (Figure 11.4a). For confidentiality, message + signature can be encrypted with a symmetric key (Figure 11.4b).

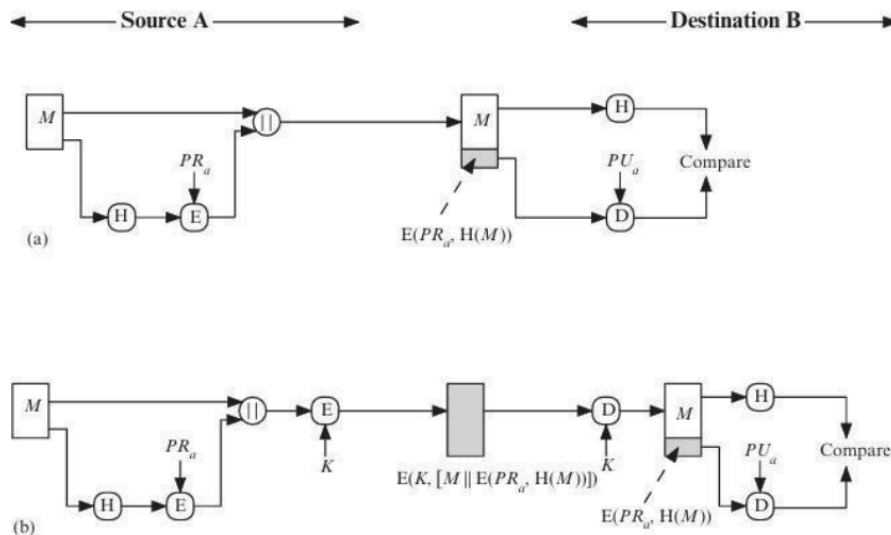


Figure 11.4 Simplified Examples of Digital Signatures

Figure 11.4: Simplified examples of digital signatures (Module 3 notes)

How integrity and authenticity are verified

1. **Integrity:** the receiver recomputes the hash of the received message and compares it with the received (decrypted) hash value. Equal values show the message was not modified, inserted or deleted in transit.
2. **Authenticity (MAC):** only the holder of the shared key K (or shared secret S) can produce a valid MAC / $H(M || S)$, so a matching value shows the message came from the other key holder.
3. **Authenticity (digital signature):** the hash decrypts correctly with the sender's public key only if it was encrypted with the sender's private key, so it proves the sender's identity and prevents repudiation.

Other applications

- **Password protection:** store the hash of passwords instead of the actual passwords; at login, the computed hash is compared with the stored hash.
- **Intrusion and virus detection:** store a secure $H(F)$ for every file; later recompute and compare to detect modification. An intruder cannot alter a file without changing its hash value.

Q.7. Explain the role of a Key Distribution Centre (KDC) in user authentication and secure key distribution with a neat diagram. Describe the authentication process and the security issues associated with KDC.

Answer:

Need for key distribution

For symmetric encryption to work, the two parties must share the same key, and that key must be protected from access by others. Key distribution is the means of delivering a key to two parties who wish to exchange data without allowing others to see the key. For parties A and B this can be done in the following ways:

1. A can select a key and physically deliver it to B.
2. A third party can select the key and physically deliver it to A and B.
3. If A and B have previously and recently used a key, one party can transmit the new key to the other, encrypted using the old key.
4. If A and B each have an encrypted connection to a third party C, C can deliver a key on the encrypted links to A and B.

Options 1 and 2 need personal contact and do not scale as the number of parties grows; option 3 depends on 1 or 2 occurring first. A **trusted third party** (option 4) is the only practical solution as the number of parties grows.

Key Distribution Centre (KDC) and key hierarchy

- A KDC is based on a **hierarchy of keys**; at a minimum two levels are used.
- Communication between end systems is encrypted using a temporary key, the **session key**, typically used for the duration of a logical connection and then discarded.
- The **master key** is shared by the KDC and an end system or user and is used to encrypt the session key.

Key distribution scenario

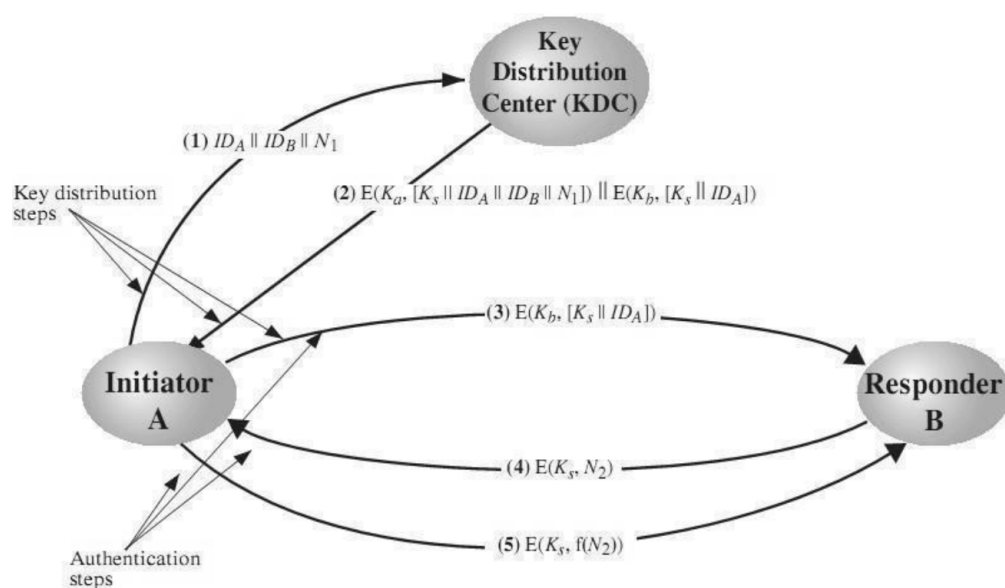


Figure 14.3 Key Distribution Scenario

Figure 14.3: Key Distribution Scenario (Module 3 notes)

A wishes to establish a logical connection with B and needs a one-time session key. A has a master key K_a known only to itself and the KDC; B shares master key K_b with the KDC.

1. **A → KDC:** $ID_A \parallel ID_B \parallel N_1$. A requests a session key for a connection to B; the message carries the identities of A and B and a unique identifier N_1 (a **nonce**: a timestamp, counter or random number; the minimum requirement is that it differs with each request, and a random number is a good choice since it is hard to guess, preventing masquerade).
2. **KDC → A:** $E(K_a, [K_s \parallel ID_A \parallel ID_B \parallel N_1]) \parallel E(K_b, [K_s \parallel ID_A])$. The message is encrypted with K_a , so only A can read it and A knows it came from the KDC. It contains the one-time session key K_s and the original request with N_1 (so A can verify that its request was not altered and that this is not a replay), plus two items for B, encrypted with K_b : K_s and ID_A .
3. **A → B:** $E(K_b, [K_s \parallel ID_A])$. A stores K_s and forwards to B the information meant for B. Being encrypted with K_b , it is protected from eavesdropping. B now knows K_s , knows that the other party is A (from ID_A), and that the information originated at the KDC.
4. **B → A:** $E(K_s, N_2)$. Using the new session key, B sends a nonce N_2 to A.
5. **A → B:** $E(K_s, f(N_2))$. A responds with $f(N_2)$, where f is a function that transforms N_2 (e.g. adding one).

The actual key distribution involves only steps 1 to 3; **steps 4 and 5, together with step 3, perform the authentication function**, assuring B that the message received in step 3 was not a replay.

Security issues and related aspects of the KDC

- **Trust:** the intermediary must be trusted not to abuse its knowledge of all session keys, and the KDC must be protected from subversion. This requirement can be avoided by fully decentralized key distribution.
- **Hierarchical key control:** for very large networks a single KDC may not be practical, so a hierarchy of KDCs can be used: local KDCs each serve a small domain (a LAN or a building); if two entities in different domains want a shared key, the local KDCs communicate through a global KDC. This minimizes the effort of master key distribution, since most master keys are shared by a local KDC with its local entities.
- **Session key lifetime:** distributing session keys delays the start of any exchange and burdens the network, so security managers must balance the two. For connection-oriented protocols use the same session key while the connection is open (and, for very long connections, change it periodically, e.g. when the PDU sequence number cycles). For connectionless protocols the most secure approach is a new key for each exchange, but a better strategy is to use a key for a fixed period or a fixed number of transactions.

A transparent key control scheme

This is useful for providing end-to-end encryption at a network or transport level in a way that is transparent to end users, assuming a connection-oriented end-to-end protocol such as TCP. Its key element is the **session security module (SSM)**, which performs end-to-end encryption and obtains session keys on behalf of its host or terminal.

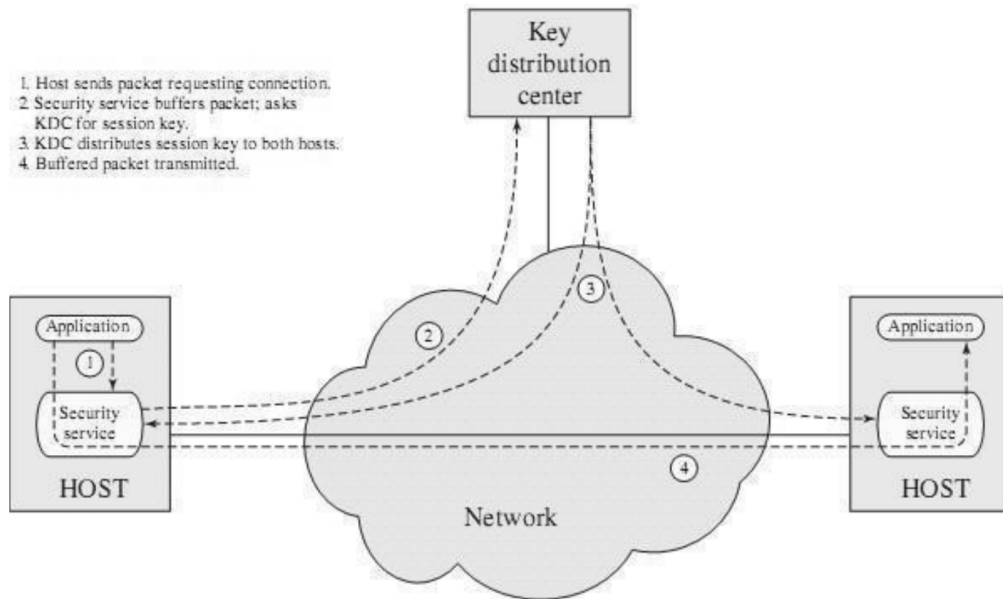


Figure 14.4 Automatic Key Distribution for Connection-Oriented Protocol

Figure 14.4: Automatic key distribution for connection-oriented protocol (Module 3 notes)

1. The host transmits a connection-request packet.
2. The SSM saves the packet and applies to the KDC for permission to establish the connection.
3. SSM–KDC communication is encrypted with a master key shared only by that SSM and the KDC. If the KDC approves, it generates the session key and delivers it to the two SSMs using a unique permanent key for each SSM.
4. The requesting SSM releases the connection-request packet and the connection is set up.
5. All user data exchanged are encrypted by the SSMs using the one-time session key.

Decentralized key control

Full decentralization avoids the need to trust a KDC, but it is not practical for larger networks using symmetric encryption only (it may be useful in a local context): each end system must be able to communicate securely with all potential partners, so as many as $n(n - 1)/2$ **master keys** may be needed for n end systems. A session key is established as follows:

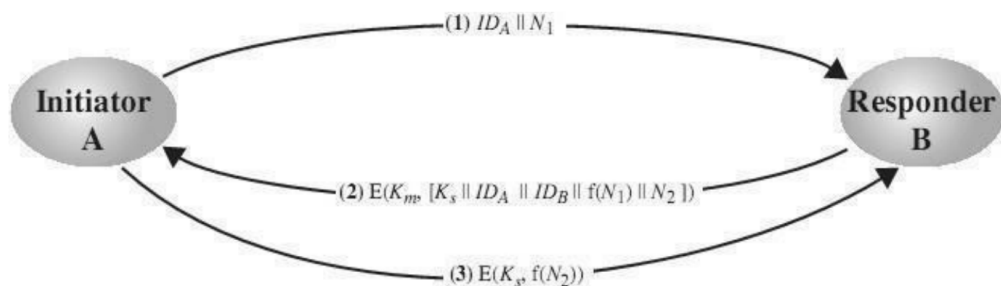


Figure 14.5 Decentralized Key Distribution

Figure 14.5: Decentralized key distribution (Module 3 notes)

1. A issues a request to B for a session key and includes a nonce N_1 : $ID_A || N_1$.
2. B responds with a message encrypted with the shared master key K_m , containing the session key K_s selected by B, the identifiers of A and B, $f(N_1)$ and another nonce N_2 .
3. Using the new session key, A returns $f(N_2)$ to B.

Q.8. Explain how Control Vector Encryption and Decryption are used to control key usage and enforce access control with a suitable diagram. Describe the process and its advantages.

Answer:

Need to control key usage

A key hierarchy and automated key distribution greatly reduce the number of keys that must be managed manually. It is also desirable to control the way automatically distributed keys are used. Besides separating master keys from session keys, different types of session keys can be defined on the basis of use:

- **Data-encrypting key**, for general communication across a network.
- **PIN-encrypting key**, for personal identification numbers used in electronic funds transfer and point-of-sale applications.
- **File-encrypting key**, for encrypting files stored in publicly accessible locations.

The value of separating keys by type: normally the master key is physically secured within the cryptographic hardware of the KDC and end systems, while session keys encrypted with it are available to application programs. If a master key were treated as a session key (for example, imported as a data-encrypting key), an unauthorized application might obtain the plaintext of session keys encrypted with that master key.

Earlier technique: the 8-bit key tag (DES)

Each 64-bit DES key has eight non-key bits normally reserved for parity checking; these form the **key tag**: one bit indicates session key or master key, one indicates whether the key can be used for encryption, one whether it can be used for decryption, and the remaining bits are spare. The tag is embedded in the key, so it is encrypted along with the key when it is distributed. Its drawbacks are: the tag length is limited to 8 bits, limiting flexibility and functionality; and because the tag is not transmitted in clear form, it can be used only at the point of decryption, limiting the ways key use can be controlled.

Control vector scheme

Each session key has an associated **control vector (CV)** consisting of fields that specify the uses and restrictions of that key; its length may vary. The CV is cryptographically coupled with the key at the time of key generation at the KDC.

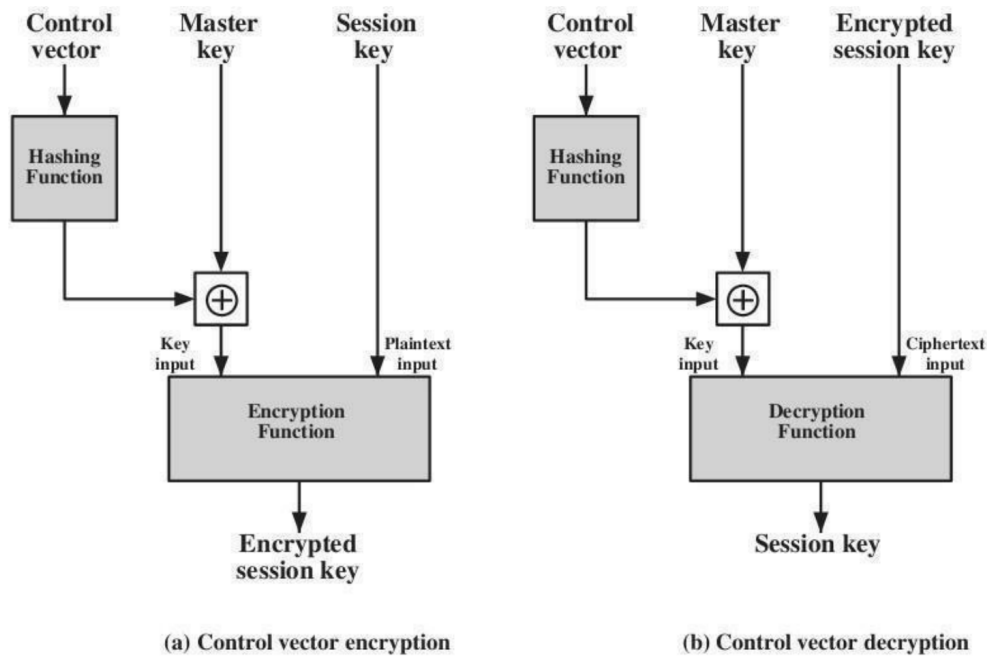


Figure 14.6 Control Vector Encryption and Decryption

Figure 14.6: Control vector encryption and decryption (Module 3 notes)

Process

(a) Encryption (at the KDC):

1. The control vector is passed through a hash function producing a value whose length equals the encryption key length: **Hash value $H = h(CV)$** . (A hash function maps values from a larger range to a smaller range with a reasonably uniform spread.)
2. The hash value is XORed with the master key to give the key input: **Key input = $K_m \text{ XOR } H$** .
3. The session key is encrypted with this key input: **Ciphertext = $E([K_m \text{ XOR } H], K_s)$** , where K_m is the master key and K_s is the session key.

(b) Decryption (at the user): the session key is recovered by the reverse operation: **$D([K_m \text{ XOR } H], E([K_m \text{ XOR } H], K_s)) = K_s$** .

When a session key is delivered to a user from the KDC, it is accompanied by the control vector **in clear form**. The session key can be recovered only by using **both** the master key the user shares with the KDC **and** the control vector, so the linkage between the session key and its CV is maintained. If the CV is altered, the hash value and hence the key input change, and the session key is not recovered correctly. This enforces the uses and restrictions specified by the CV.

Advantages over the 8-bit tag

1. There is **no restriction on the length** of the control vector, which enables arbitrarily complex controls to be imposed on key use.
2. The control vector is **available in clear form at all stages of operation**, so control of key use can be exercised in multiple locations.

Q.9. Explain the general elements and format of an X.509 certificate. Describe the architecture of the X.509 Public Key Infrastructure (PKI) model with a neat diagram.

Answer:

X.509

X.509 is part of the X.500 series of recommendations that define a directory service (a server or distributed set of servers that maintains a database of information about users). X.509 defines a framework for authentication services provided by the X.500 directory to its users; the directory may serve as a repository of public-key certificates. Each certificate contains the public key of a user and is signed with the private key of a trusted **certification authority (CA)**. X.509 is based on public-key cryptography and digital signatures, and the format is widely used in S/MIME, IP Security, SSL/TLS and SET. It was first issued in 1988, revised in 1993, a third version was issued in 1995 and revised in 2000.

User certificates are created by a trusted CA and placed in the directory by the CA or the user. The directory server is not responsible for creating public keys or for certification; it merely provides an accessible location for certificates. The notation for the certificate of A issued by CA is $CA\langle\langle A \rangle\rangle = CA\{V, SN, AI, CA, U_{CA}, A, U_A, A_p, T_A\}$.

General elements and format of an X.509 certificate

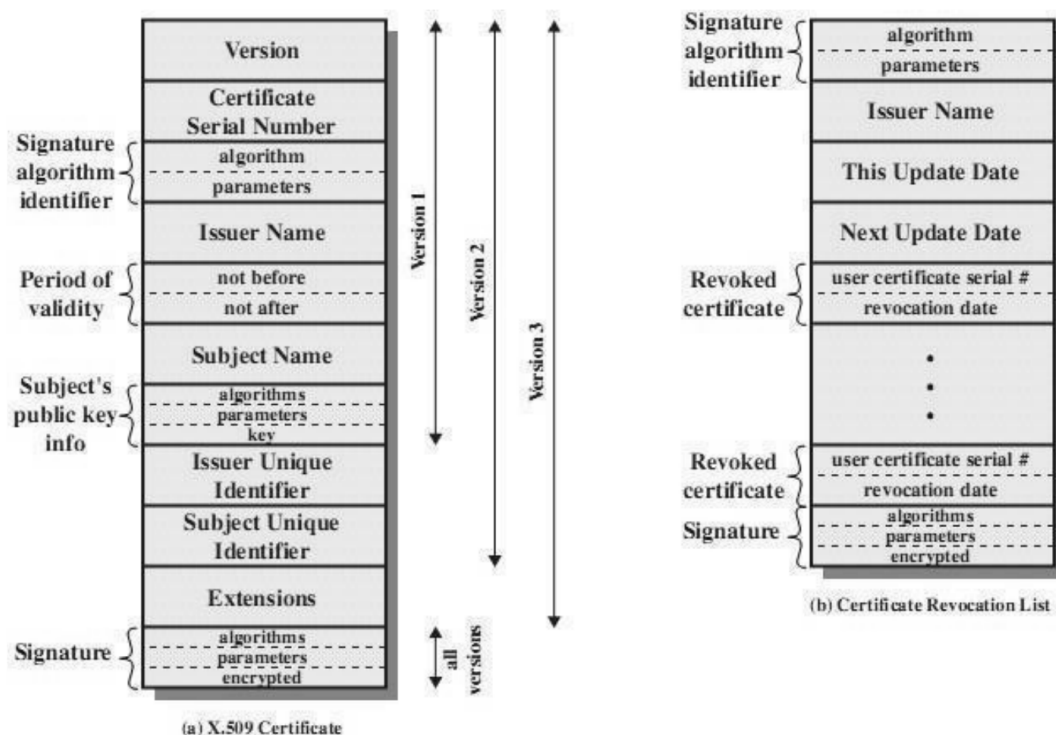


Figure 14.4 X.509 Formats

Figure 14.4: X.509 formats – (a) X.509 certificate, (b) Certificate revocation list (Module 3 notes)

Field	Meaning
Version (V)	Differentiates among successive versions of the certificate format; default is version 1. If the issuer or subject unique identifier is present, the value must be version 2; if one or more extensions are present, it must be version 3.
Serial number (SN)	An integer value unique within the issuing CA, unambiguously associated with this certificate.
Signature algorithm identifier (AI)	The algorithm used to sign the certificate, together with any parameters. It is repeated in the signature field at the end, so it has little utility.
Issuer name	X.500 name of the CA that created and signed this certificate.
Period of validity (T_A)	Two dates: the first and last on which the certificate is valid.
Subject name	The name of the user to whom the certificate refers, i.e. it certifies the public key of the subject who holds the corresponding private key.
Subject's public-key information	The public key of the subject, plus an identifier of the algorithm for which the key is to be used, with any parameters.
Issuer unique identifier	Optional bit-string field to identify the issuing CA uniquely if the X.500 name has been reused for different entities.
Subject unique identifier	Optional bit-string field to identify the subject uniquely if the X.500 name has been reused for different entities.
Extensions	One or more extension fields (added in version 3).
Signature	Covers all other fields; it contains the hash code of the other fields encrypted with the CA's private key, and includes the signature algorithm identifier.

The unique identifier fields were added in version 2 to handle the possible reuse of subject and/or issuer names over time; they are rarely used. Notation: $Y\langle\langle X \rangle\rangle$ = certificate of user X issued by CA Y; $Y\{I\}$ = signing of I by Y (I with an encrypted hash code appended); V = version; SN = serial number; AI = algorithm identifier; CA = name of CA; U_{CA} = unique identifier of CA; A = name of user A; U_A = unique identifier of user A; A_p = public key of A; T_A = period of validity.

Certificate revocation

A certificate may need to be revoked before it expires if the user's private key is assumed compromised, the user is no longer certified by the CA, or the CA's certificate is assumed compromised. Each CA maintains a **certificate revocation list (CRL)** of revoked but not expired certificates. Each CRL is signed by the issuer and includes the issuer's name, the date the list was created, the date the next CRL is due, and an entry (certificate serial number and revocation date) for each revoked certificate (Figure 14.4b).

Public Key Infrastructure (PKI) architecture

A PKI is the set of hardware, software, people, policies and procedures needed to create, manage, store, distribute and revoke digital certificates based on asymmetric cryptography. Its principal objective is to enable secure, convenient and efficient acquisition of public keys.

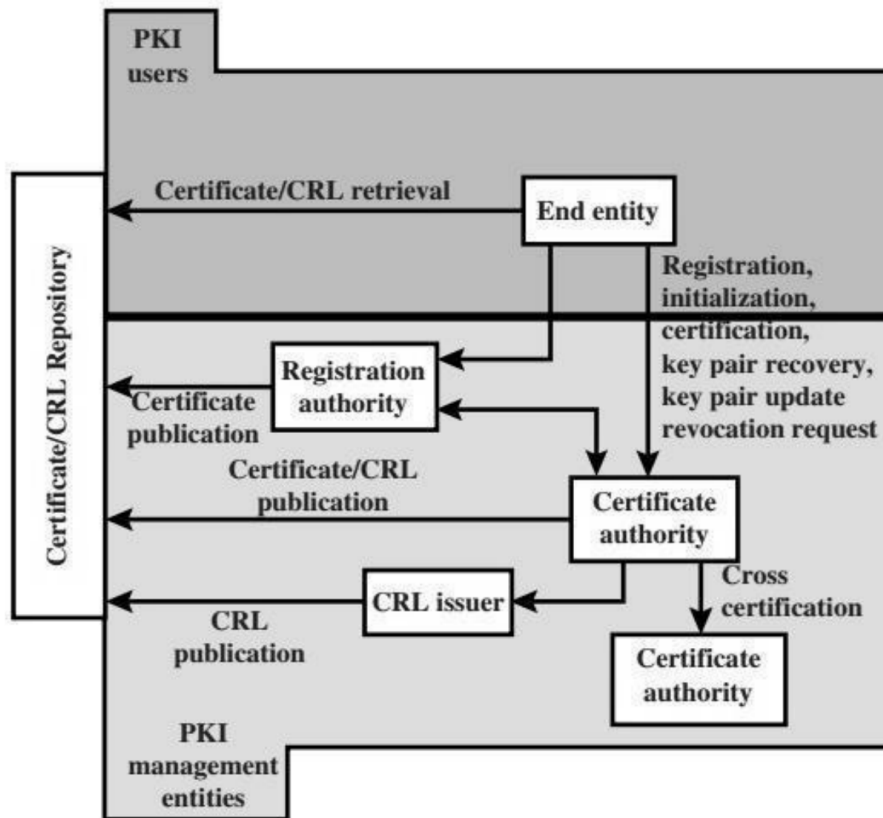


Figure: PKIX architectural model (Module 3 notes)

- **End entity:** end users, devices (e.g. servers, routers) or any other entity that can be identified in the subject field of a public-key certificate.
- **Certification authority (CA):** the issuer of certificates and certificate revocation lists (CRLs).
- **Registration authority (RA):** an optional component that can assume a number of administrative functions from the CA.
- **CRL issuer:** an optional component that a CA can delegate to publish CRLs.
- **Repository:** any method for storing certificates and CRLs so that they can be retrieved by end entities.

PKIX management functions: *Registration* (a user first makes itself known to a CA, directly or through an RA); *Initialization* (installing key material, e.g. the public key of the trusted CA); *Certification* (the CA issues a certificate for the user's public key and returns it and/or posts it in a repository); *Key pair recovery* (restoring the encryption/decryption key pair from an authorized backup facility); *Key pair update* (regular replacement of key pairs with new certificates); *Revocation request* (an authorized person advises the CA of a situation requiring revocation); *Cross certification* (a certificate issued by one CA to another CA containing a CA signature key used for issuing certificates).

PKIX management protocols: Certificate Management Protocols (CMP), where each management function is explicitly identified by specific protocol exchanges, and Certificate Management Messages over CMS (CMC), which is built on earlier work to leverage existing implementations and can be used to obtain X.509 certificates in a PKI.

Q.10. Explain the various schemes of public key distribution in detail. Discuss their working principles, advantages, and limitations with suitable diagrams.

Answer:

Several techniques have been proposed for the distribution of public keys. They can mostly be grouped into: (1) public announcement, (2) publicly available directory, (3) public-key authority, and (4) public-key certificates.

1. Public announcement of public keys

The point of public-key encryption is that the public key is public, so any participant can send his or her public key to any other participant, or broadcast it to the community at large, e.g. appending PGP keys to e-mail messages or posting them to newsgroups or e-mail lists.



Figure 10.1 Uncontrolled Public Key Distribution

Figure 10.1: Uncontrolled public key distribution (Module 3 notes)

- **Advantage:** very simple and convenient.
- **Limitation:** its major weakness is **forgery**. Anyone could pretend to be user A and send a public key to another participant or broadcast such a key. Until the forgery is discovered they can masquerade as the claimed user.

2. Publicly available directory

Greater security is obtained by registering keys with a public directory, which must be trusted and has these properties:

- The authority maintains a directory with a {name, public key} entry for each participant.
- Each participant registers a public key with the directory authority.
- A participant may replace the existing key with a new one at any time (e.g. because the private key was compromised).
- Participants can also access the directory electronically; for this, secure, authenticated communication from the authority to the participant is mandatory.

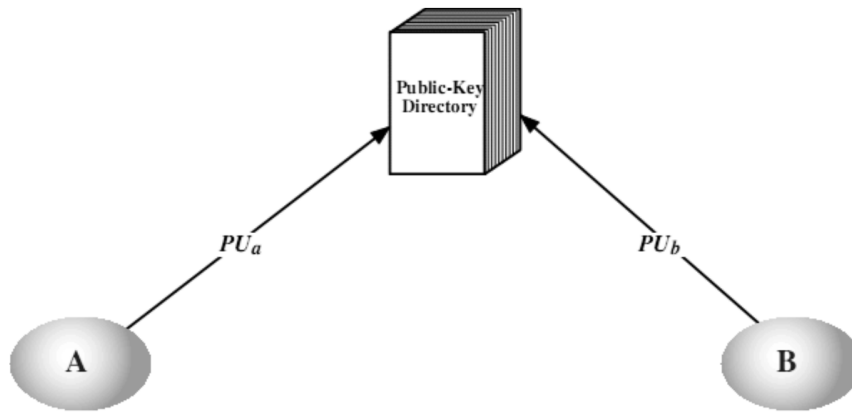


Figure 10.2 Public Key Publication

Figure 10.2: Public key publication (Module 3 notes)

- **Advantage:** more secure than individual public announcements.
- **Limitation:** if an adversary obtains or computes the directory authority's private key, he can authoritatively pass out counterfeit public keys, impersonate any participant and eavesdrop on messages sent to any participant. The same can be achieved by tampering with the records kept by the authority.

3. Public-key authority

Stronger security is achieved by tighter control over the distribution of public keys from the directory. It requires users to know the public key of the authority and to interact with it in real time to obtain any desired public key securely. A total of **seven messages** are required.

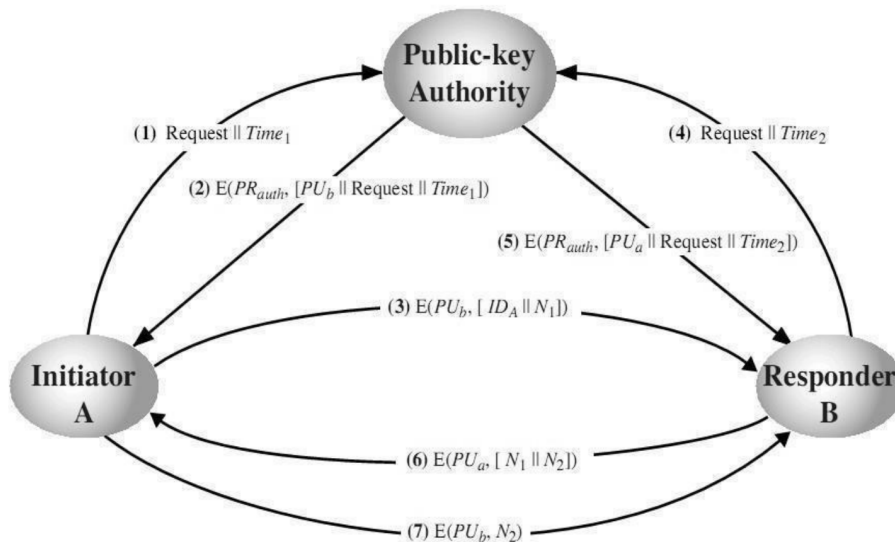


Figure: Public-key distribution scenario using a public-key authority (Module 3 notes)

1. A sends a timestamped message to the public-key authority requesting the current public key of B.
2. The authority responds with a message encrypted with its private key PR_{auth} , so A can decrypt it with the authority's public key and is assured it came from the authority. The message includes B's public key PU_b , the original request (so A can match the response and verify the request was not altered), and the original timestamp (so A knows it is not an old message carrying a key other than

B's current one).

3. A stores PU_b and uses it to encrypt a message to B containing ID_A and a nonce N_1 .
 4. B retrieves A's public key from the authority in the same manner (messages 4 and 5).
 5. B sends A a message encrypted with PU_a containing N_1 and a new nonce N_2 . The presence of N_1 assures A that the correspondent is B.
 6. A returns N_2 , encrypted with B's public key, assuring B that its correspondent is A.
- **Advantage:** stronger than the directory scheme; the authority's signature makes each key response verifiable.
 - **Limitation:** the authority is a bottleneck, since a user must appeal to it for the public key of every other user it wishes to contact; it must be accessed in real time; and the directory of names and keys is still vulnerable to tampering.

4. Public-key certificates

Certificates allow keys to be exchanged **without contacting a public-key authority**. A certificate binds an identity to a public key, with all contents signed by a trusted public-key or Certificate Authority (CA). It can be verified by anyone who knows the CA's public key, and a participant can convey its key information to another by transmitting its certificate. The requirements of the scheme are:

1. Any participant can read a certificate to determine the name and public key of its owner.
2. Any participant can verify that the certificate originated from the CA and is not counterfeit.
3. Only the CA can create and update certificates.
4. Any participant can verify the currency of the certificate.

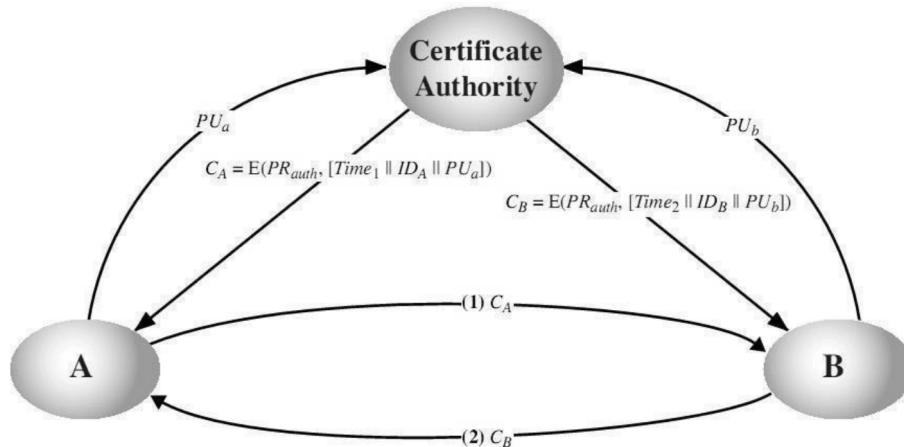


Figure: Exchange of public-key certificates (Module 3 notes)

Each participant applies to the CA, supplying a public key, and obtains a certificate $C_A = E(PR_{auth}, [Time_1 || ID_A || PU_a])$ and $C_B = E(PR_{auth}, [Time_2 || ID_B || PU_b])$. A and B exchange certificates (1) C_A , (2) C_B , and each verifies the other's using the CA's public key. The scheme that has become universally accepted for formatting public-key certificates is **X.509** (Q.9), used in IP security, SSL, SET and S/MIME.

- **Advantage:** no real-time access to an authority is needed; certificates are unforgeable, so they can be placed in a directory without special protection.
- **Limitation:** all users must trust the CA and know its public key; if users belong to different CAs, a chain (hierarchy) of certificates is required, and revoked certificates must be checked against the CRL.

Summary comparison

Scheme	Principle	Main limitation
Public announcement	Owner broadcasts own public key	Forgery / masquerade
Public directory	Trusted directory holds {name, key} entries	Directory authority's private key or records can be compromised
Public-key authority	Authority supplies signed keys on request (7 messages)	Bottleneck; real-time access needed; directory can be tampered
Public-key certificates	CA-signed certificate binds identity to key	Requires trusted CA(s), CA hierarchy and CRL checking

Q.11. Let $p = 29$ and $g = 2$. If the private keys of users A and B are 5 and 12, respectively, use the Diffie–Hellman key exchange algorithm to calculate their public keys and the common shared secret key, showing the necessary steps.

Answer:

Given: prime $q (p) = 29$, primitive root $\alpha (g) = 2$, private key of A (X_A) = 5, private key of B (X_B) = 12.

Step 1: Public key of A ($Y_A = \alpha^{X_A} \bmod q$)

$$Y_A = 2^5 \bmod 29$$

$$2^1 \bmod 29 = 2$$

$$2^2 = (2)^2 \bmod 29 = 4$$

$$2^4 = (4)^2 \bmod 29 = 16$$

Since $5 = 4 + 1$:

$$2^5 = 16 \times 2 \bmod 29$$

$$16 \times 2 = 32 \bmod 29 = 3$$

$$Y_A = 3$$

Step 2: Public key of B ($Y_B = \alpha^{X_B} \bmod q$)

$$Y_B = 2^{12} \bmod 29$$

$$2^1 \bmod 29 = 2$$

$$2^2 = (2)^2 \bmod 29 = 4$$

$$2^4 = (4)^2 \bmod 29 = 16$$

$$2^8 = (16)^2 \bmod 29 = 24$$

Since $12 = 8 + 4$:

$$2^{12} = 24 \times 16 \bmod 29$$

$$24 \times 16 = 384 \bmod 29 = 7$$

$$Y_B = 7$$

Step 3: Exchange. A sends $Y_A = 3$ to B and B sends $Y_B = 7$ to A (in plaintext).

Step 4: Shared secret key computed by A ($K = (Y_B)^{X_A} \bmod q$)

$$K = 7^5 \bmod 29$$

$$7^1 \bmod 29 = 7$$

$$7^2 = (7)^2 \bmod 29 = 20$$

$$7^4 = (20)^2 \bmod 29 = 23$$

Since $5 = 4 + 1$:

$$7^5 = 23 \times 7 \bmod 29$$

$$23 \times 7 = 161 \bmod 29 = 16$$

$$K = 16$$

Step 5: Shared secret key computed by B ($K = (Y_A)^{X_B} \bmod q$)

$$K = 3^{12} \bmod 29$$

$$3^1 \bmod 29 = 3$$

$$3^2 = (3)^2 \bmod 29 = 9$$

$$3^4 = (9)^2 \bmod 29 = 23$$

$$3^8 = (23)^2 \bmod 29 = 7$$

Since $12 = 8 + 4$:

$$3^{12} = 7 \times 23 \bmod 29$$

$$7 \times 23 = 161 \bmod 29 = 16$$

K = 16

Quantity	Value
Public key of A, Y_A	3
Public key of B, Y_B	7
Shared secret key K (computed by A)	16
Shared secret key K (computed by B)	16

Both users obtain the same secret key **K = 16**. An attacker sees only $q = 29$, $\alpha = 2$, $Y_A = 3$, $Y_B = 7$ but not the private keys.

Q.12. Let $p = 31$ and $g = 3$. If user A selects private key $a = 7$ and user B selects private key $b = 11$, calculate their public keys and the common shared secret key using the Diffie–Hellman key exchange algorithm. Show all intermediate calculations.

Answer:

Given: prime $q (p) = 31$, primitive root $\alpha (g) = 3$, private key of A (X_A) = 7, private key of B (X_B) = 11.

Step 1: Public key of A ($Y_A = \alpha^{X_A} \bmod q$)

$$Y_A = 3^7 \bmod 31$$

$$3^1 \bmod 31 = 3$$

$$3^2 = (3)^2 \bmod 31 = 9$$

$$3^4 = (9)^2 \bmod 31 = 19$$

Since $7 = 4 + 2 + 1$:

$$3^7 = 19 \times 9 \times 3 \pmod{31}$$

$$19 \times 9 = 171 \bmod 31 = 16$$

$$16 \times 3 = 48 \bmod 31 = 17$$

$$Y_A = 17$$

Step 2: Public key of B ($Y_B = \alpha^{X_B} \bmod q$)

$$Y_B = 3^{11} \bmod 31$$

$$3^1 \bmod 31 = 3$$

$$3^2 = (3)^2 \bmod 31 = 9$$

$$3^4 = (9)^2 \bmod 31 = 19$$

$$3^8 = (19)^2 \bmod 31 = 20$$

Since $11 = 8 + 2 + 1$:

$$3^{11} = 20 \times 9 \times 3 \pmod{31}$$

$$20 \times 9 = 180 \bmod 31 = 25$$

$$25 \times 3 = 75 \bmod 31 = 13$$

$$Y_B = 13$$

Step 3: Exchange. A sends $Y_A = 17$ to B and B sends $Y_B = 13$ to A (in plaintext).

Step 4: Shared secret key computed by A ($K = (Y_B)^{X_A} \bmod q$)

$$K = 13^7 \bmod 31$$

$$13^1 \bmod 31 = 13$$

$$13^2 = (13)^2 \bmod 31 = 14$$

$$13^4 = (14)^2 \bmod 31 = 10$$

Since $7 = 4 + 2 + 1$:

$$13^7 = 10 \times 14 \times 13 \pmod{31}$$

$$10 \times 14 = 140 \bmod 31 = 16$$

$$16 \times 13 = 208 \bmod 31 = 22$$

$$K = 22$$

Step 5: Shared secret key computed by B ($K = (Y_A)^{X_B} \bmod q$)

$$K = 17^{11} \bmod 31$$

$$17^1 \bmod 31 = 17$$

$$17^2 = (17)^2 \bmod 31 = 10$$

$$17^4 = (10)^2 \bmod 31 = 7$$

$$17^8 = (7)^2 \bmod 31 = 18$$

Since $11 = 8 + 2 + 1$:

$$17^{11} = 18 \times 10 \times 17 \pmod{31}$$

$$18 \times 10 = 180 \bmod 31 = 25$$

$$25 \times 17 = 425 \bmod 31 = 22$$

K = 22

Quantity	Value
Public key of A, Y_A	17
Public key of B, Y_B	13
Shared secret key K (computed by A)	22
Shared secret key K (computed by B)	22

Both users obtain the same secret key **K = 22**. An attacker sees only $q = 31$, $\alpha = 3$, $Y_A = 17$, $Y_B = 13$ but not the private keys.

Q.13. In a Diffie–Hellman key exchange, let $p = 17$ and $g = 3$. If the private keys of users A and B are $a = 5$ and $b = 6$, respectively, determine the public keys and the shared secret key. Show the necessary steps.

Answer:

Given: prime $q (p) = 17$, primitive root $\alpha (g) = 3$, private key of A (X_A) = 5, private key of B (X_B) = 6.

Step 1: Public key of A ($Y_A = \alpha^{X_A} \text{ mod } q$)

$$Y_A = 3^5 \text{ mod } 17$$

$$3^1 \text{ mod } 17 = 3$$

$$3^2 = (3)^2 \text{ mod } 17 = 9$$

$$3^4 = (9)^2 \text{ mod } 17 = 13$$

Since $5 = 4 + 1$:

$$3^5 = 13 \times 3 \text{ (mod } 17)$$

$$13 \times 3 = 39 \text{ mod } 17 = 5$$

$$Y_A = 5$$

Step 2: Public key of B ($Y_B = \alpha^{X_B} \text{ mod } q$)

$$Y_B = 3^6 \text{ mod } 17$$

$$3^1 \text{ mod } 17 = 3$$

$$3^2 = (3)^2 \text{ mod } 17 = 9$$

$$3^4 = (9)^2 \text{ mod } 17 = 13$$

Since $6 = 4 + 2$:

$$3^6 = 13 \times 9 \text{ (mod } 17)$$

$$13 \times 9 = 117 \text{ mod } 17 = 15$$

$$Y_B = 15$$

Step 3: Exchange. A sends $Y_A = 5$ to B and B sends $Y_B = 15$ to A (in plaintext).

Step 4: Shared secret key computed by A ($K = (Y_B)^{X_A} \text{ mod } q$)

$$K = 15^5 \text{ mod } 17$$

$$15^1 \text{ mod } 17 = 15$$

$$15^2 = (15)^2 \text{ mod } 17 = 4$$

$$15^4 = (4)^2 \text{ mod } 17 = 16$$

Since $5 = 4 + 1$:

$$15^5 = 16 \times 15 \text{ (mod } 17)$$

$$16 \times 15 = 240 \text{ mod } 17 = 2$$

$$K = 2$$

Step 5: Shared secret key computed by B ($K = (Y_A)^{X_B} \text{ mod } q$)

$$K = 5^6 \text{ mod } 17$$

$$5^1 \text{ mod } 17 = 5$$

$$5^2 = (5)^2 \text{ mod } 17 = 8$$

$$5^4 = (8)^2 \text{ mod } 17 = 13$$

Since $6 = 4 + 2$:

$$5^6 = 13 \times 8 \pmod{17}$$

$$13 \times 8 = 104 \pmod{17} = 2$$

$$\mathbf{K} = 2$$

Quantity	Value
Public key of A, Y_A	5
Public key of B, Y_B	15
Shared secret key K (computed by A)	2
Shared secret key K (computed by B)	2

Both users obtain the same secret key $\mathbf{K} = 2$. An attacker sees only $q = 17$, $\alpha = 3$, $Y_A = 5$, $Y_B = 15$ but not the private keys.

Q.14. Apply the ECDH key exchange procedure over the elliptic curve $E_{211}(0, -4)$, with generator $G = (2, 2)$. If the private keys of users A and B are $n_A = 5$ and $n_B = 7$, respectively, calculate their public keys and the common shared secret key, showing the necessary elliptic curve point calculations.

Answer:

Given: elliptic curve $E_{211}(0, -4)$, i.e. $y^2 \equiv x^3 - 4 \pmod{211}$ with $a = 0$, $b = -4$, $q = p = 211$, base point $G = (2, 2)$. Private keys: $n_A = 5$, $n_B = 7$.

Check that G lies on the curve: $y^2 = 2^2 = 4$ and $x^3 - 4 = 8 - 4 = 4$. Since both sides are equal, $G = (2, 2)$ is a valid point.

Point addition rules used (mod 211, $a = 0$):

- For $P = (x_1, y_1)$, $Q = (x_2, y_2)$, $P \neq Q$: $\lambda = (y_2 - y_1)/(x_2 - x_1) \pmod{211}$.
- For doubling ($P = Q$): $\lambda = (3x_1^2 + a)/(2y_1) \pmod{211}$, with $a = 0$.
- Then $x_3 = \lambda^2 - x_1 - x_2 \pmod{211}$ and $y_3 = \lambda(x_1 - x_3) - y_1 \pmod{211}$.
- Scalar multiplication kG is done by the double-and-add method: write k in binary, start from G (leading bit); for every next bit, double the current point and, if the bit is 1, add G .

Step 1: Public key of A, $P_A = n_A \times G = 5G$. Binary of 5 = 101.

Step	Operation	$\lambda \pmod{211}$	$x_3 = \lambda^2 - x_1 - x_2$	$y_3 = \lambda(x_1 - x_3) - y_1$	Result
1	$2G = 2 \times 1G$ (2, 2) doubled	$(3 \cdot 2^2)/(2 \cdot 2) = 12/4 \equiv 12/4 \equiv 3$	$3^2 - 2 - 2 = 5 \equiv 5$	$3(2 - 5) - 2 = -11 \equiv 200$	$2G = (5, 200)$
2	$4G = 2 \times 2G$ (5, 200) doubled	$(3 \cdot 5^2)/(2 \cdot 200) = 75/400 \equiv 75/189 \equiv 198$	$198^2 - 5 - 5 = 39194 \equiv 159$	$198(5 - 159) - 200 = -30692 \equiv 114$	$4G = (159, 114)$
3	$5G = 4G + G$ (159, 114) + (2, 2)	$(2 - 114)/(2 - 159) = -112/-157 \equiv 99/54 \equiv 37$	$37^2 - 159 - 2 = 1208 \equiv 153$	$37(159 - 153) - 114 = 108 \equiv 108$	$5G = (153, 108)$

$P_A = 5G = (153, 108)$

Step 2: Public key of B, $P_B = n_B \times G = 7G$. Binary of 7 = 111.

Step	Operation	$\lambda \pmod{211}$	$x_3 = \lambda^2 - x_1 - x_2$	$y_3 = \lambda(x_1 - x_3) - y_1$	Result
1	$2G = 2 \times 1G$ (2, 2) doubled	$(3 \cdot 2^2)/(2 \cdot 2) = 12/4 \equiv 12/4 \equiv 3$	$3^2 - 2 - 2 = 5 \equiv 5$	$3(2 - 5) - 2 = -11 \equiv 200$	$2G = (5, 200)$
2	$3G = 2G + G$ (5, 200) + (2, 2)	$(2 - 200)/(2 - 5) = -198/-3 \equiv 13/208 \equiv 66$	$66^2 - 5 - 2 = 4349 \equiv 129$	$66(5 - 129) - 200 = -8384 \equiv 56$	$3G = (129, 56)$
3	$6G = 2 \times 3G$ (129, 56) doubled	$(3 \cdot 129^2)/(2 \cdot 56) = 49923/112 \equiv 127/112 \equiv 52$	$52^2 - 129 - 129 = 2446 \equiv 125$	$52(129 - 125) - 56 = 152 \equiv 152$	$6G = (125, 152)$
4	$7G = 6G + G$ (125, 152) + (2, 2)	$(2 - 152)/(2 - 125) = -150/-123 \equiv 61/88 \equiv 99$	$99^2 - 125 - 2 = 9674 \equiv 179$	$99(125 - 179) - 152 = -5498 \equiv 199$	$7G = (179, 199)$

$P_B = 7G = (179, 199)$

Step 3: A and B exchange P_A and P_B .

Step 4: Secret key computed by A, $K = n_A \times P_B = 5 \times (179, 199)$. (Binary of 5 = 101; the base point for double-and-add is now P_B .)

Step	Operation	$\lambda \pmod{211}$	$x_3 = \lambda^2 - x_1 - x_2$	$y_3 = \lambda(x_1 - x_2) - y_1$	Result
1	$2P_B = 2 \times 1P_B$ (179, 199) doubled	$(3 \cdot 179^2)/(2 \cdot 199) =$ $96123/398 \equiv 118/187 \equiv \mathbf{83}$	$83^2 - 179 - 179 =$ $6531 \equiv 201$	$83(179 - 201) - 199 = -2025$ $\equiv 85$	$2P_B = (201, 85)$
2	$4P_B = 2 \times 2P_B$ (201, 85) doubled	$(3 \cdot 201^2)/(2 \cdot 85) =$ $121203/170 \equiv 89/170 \equiv \mathbf{39}$	$39^2 - 201 - 201 =$ $1119 \equiv 64$	$39(201 - 64) - 85 = 5258$ $\equiv 194$	$4P_B = (64, 194)$
3	$5P_B = 4P_B + P_B$ (64, 194) + (179, 199)	$(199 - 194)/(179 - 64) =$ $5/115 \equiv 5/115 \equiv \mathbf{156}$	$156^2 - 64 - 179 =$ $24093 \equiv 39$	$156(64 - 39) - 194 = 3706$ $\equiv 119$	$5P_B = (39, 119)$

$K = (39, 119)$

Step 5: Secret key computed by B, $K = n_B \times P_A = 7 \times (153, 108)$. The same double-and-add procedure (binary of 7 = 111) gives the successive points:

$$2P_A = (75, 90); 3P_A = (28, 2); 6P_A = (70, 200); 7P_A = (39, 119)$$

$K = (39, 119)$

This is the same point, because $n_A \times P_B = n_A \times (n_B \times G) = n_B \times (n_A \times G) = n_B \times P_A$.

Quantity	Value
$P_A = n_A G$	(153, 108)
$P_B = n_B G$	(179, 199)
$K = n_A \times P_B$	(39, 119)
$K = n_B \times P_A$	(39, 119)

Common shared secret key $K = (39, 119)$. (Usually the x-coordinate, 39, is used as the key material.)

Q.15. Consider the elliptic curve $E_{211}(0, -4)$ with generator $G = (2, 2)$. Using the ECDH algorithm, determine the public keys and shared secret key for users A and B with private keys $n_A = 3$ and $n_B = 9$, respectively. Show the point multiplication and shared-key computation steps.

Answer:

Given: elliptic curve $E_{211}(0, -4)$, i.e. $y^2 \equiv x^3 - 4 \pmod{211}$ with $a = 0$, $b = -4$, $q = p = 211$, base point $G = (2, 2)$. Private keys: $n_A = 3$, $n_B = 9$.

Check that G lies on the curve: $y^2 = 2^2 = 4$ and $x^3 - 4 = 8 - 4 = 4$. Since both sides are equal, $G = (2, 2)$ is a valid point.

Point addition rules used (mod 211, $a = 0$):

- For $P = (x_1, y_1)$, $Q = (x_2, y_2)$, $P \neq Q$: $\lambda = (y_2 - y_1)/(x_2 - x_1) \pmod{211}$.
- For doubling ($P = Q$): $\lambda = (3x_1^2 + a)/(2y_1) \pmod{211}$, with $a = 0$.
- Then $x_3 = \lambda^2 - x_1 - x_2 \pmod{211}$ and $y_3 = \lambda(x_1 - x_3) - y_1 \pmod{211}$.
- Scalar multiplication kG is done by the double-and-add method: write k in binary, start from G (leading bit); for every next bit, double the current point and, if the bit is 1, add G .

Step 1: Public key of A, $P_A = n_A \times G = 3G$. Binary of 3 = 11.

Step	Operation	$\lambda \pmod{211}$	$x_3 = \lambda^2 - x_1 - x_2$	$y_3 = \lambda(x_1 - x_3) - y_1$	Result
1	$2G = 2 \times 1G$ (2, 2) doubled	$(3 \cdot 2^2)/(2 \cdot 2) = 12/4 \equiv 12/4 \equiv 3$	$3^2 - 2 - 2 = 5 \equiv 5$	$3(2 - 5) - 2 = -11 \equiv 200$	$2G = (5, 200)$
2	$3G = 2G + G$ (5, 200) + (2, 2)	$(2 - 200)/(2 - 5) = -198/-3 \equiv 13/208 \equiv 66$	$66^2 - 5 - 2 = 4349 \equiv 129$	$66(5 - 129) - 200 = -8384 \equiv 56$	$3G = (129, 56)$

$P_A = 3G = (129, 56)$

Step 2: Public key of B, $P_B = n_B \times G = 9G$. Binary of 9 = 1001.

Step	Operation	$\lambda \pmod{211}$	$x_3 = \lambda^2 - x_1 - x_2$	$y_3 = \lambda(x_1 - x_3) - y_1$	Result
1	$2G = 2 \times 1G$ (2, 2) doubled	$(3 \cdot 2^2)/(2 \cdot 2) = 12/4 \equiv 12/4 \equiv 3$	$3^2 - 2 - 2 = 5 \equiv 5$	$3(2 - 5) - 2 = -11 \equiv 200$	$2G = (5, 200)$
2	$4G = 2 \times 2G$ (5, 200) doubled	$(3 \cdot 5^2)/(2 \cdot 200) = 75/400 \equiv 75/189 \equiv 198$	$198^2 - 5 - 5 = 39194 \equiv 159$	$198(5 - 159) - 200 = -30692 \equiv 114$	$4G = (159, 114)$
3	$8G = 2 \times 4G$ (159, 114) doubled	$(3 \cdot 159^2)/(2 \cdot 114) = 75843/228 \equiv 94/17 \equiv 80$	$80^2 - 159 - 159 = 6082 \equiv 174$	$80(159 - 174) - 114 = -1314 \equiv 163$	$8G = (174, 163)$
4	$9G = 8G + G$ (174, 163) + (2, 2)	$(2 - 163)/(2 - 174) = -161/-172 \equiv 50/39 \equiv 169$	$169^2 - 174 - 2 = 28385 \equiv 111$	$169(174 - 111) - 163 = 10484 \equiv 145$	$9G = (111, 145)$

$P_B = 9G = (111, 145)$

Step 3: A and B exchange P_A and P_B .

Step 4: Secret key computed by A, $K = n_A \times P_B = 3 \times (111, 145)$. (Binary of 3 = 11; the base point for double-and-add is now P_B .)

Step	Operation	$\lambda \pmod{211}$	$x_3 = \lambda^2 - x_1 - x_2$	$y_3 = \lambda(x_1 - x_2) - y_1$	Result
1	$2P_B = 2 \times 1P_B$ (111, 145) doubled	$(3 \cdot 111^2)/(2 \cdot 145) =$ $36963/290 \equiv 38/79 \equiv \mathbf{118}$	$118^2 - 111 - 111 =$ $13702 \equiv 198$	$118(111 - 198) - 145 =$ $-10411 \equiv 139$	$2P_B = (\mathbf{198}, \mathbf{139})$
2	$3P_B = 2P_B + P_B$ (198, 139) + (111, 145)	$(145-139)/(111-198) =$ $6/-87 \equiv 6/124 \equiv \mathbf{160}$	$160^2 - 198 - 111 =$ $25291 \equiv 182$	$160(198 - 182) - 139 =$ $2421 \equiv 100$	$3P_B = (\mathbf{182}, \mathbf{100})$

K = (182, 100)

Step 5: Secret key computed by B, $K = n_B \times P_A = 9 \times (129, 56)$. The same double-and-add procedure (binary of 9 = 1001) gives the successive points:

$$2P_A = (125, 152); 4P_A = (155, 96); 8P_A = (192, 201); 9P_A = (182, 100)$$

K = (182, 100)

This is the same point, because $n_A \times P_B = n_A \times (n_B \times G) = n_B \times (n_A \times G) = n_B \times P_A$.

Quantity	Value
$P_A = n_A G$	(129, 56)
$P_B = n_B G$	(111, 145)
$K = n_A \times P_B$	(182, 100)
$K = n_B \times P_A$	(182, 100)

Common shared secret key K = (182, 100). (Usually the x-coordinate, 182, is used as the key material.)